

Pre-trained 모델 활용

02 이미지 분류 모델

목차

—
Tensorflow를 이용해
이미지 분류 모델을 가져와
이미지를 입력하고
어떤 물체인지 분류하는
과정을 학습합니다.

- 01 이미지 분류 모델
- 02 데이터 전처리
- 03 추론 결과 처리하기

01

이미지 분류 모델

④ 이미지 분류 모델이란?

이미지를 넣으면 어떤 클래스에 속하는지 예측하는 모델

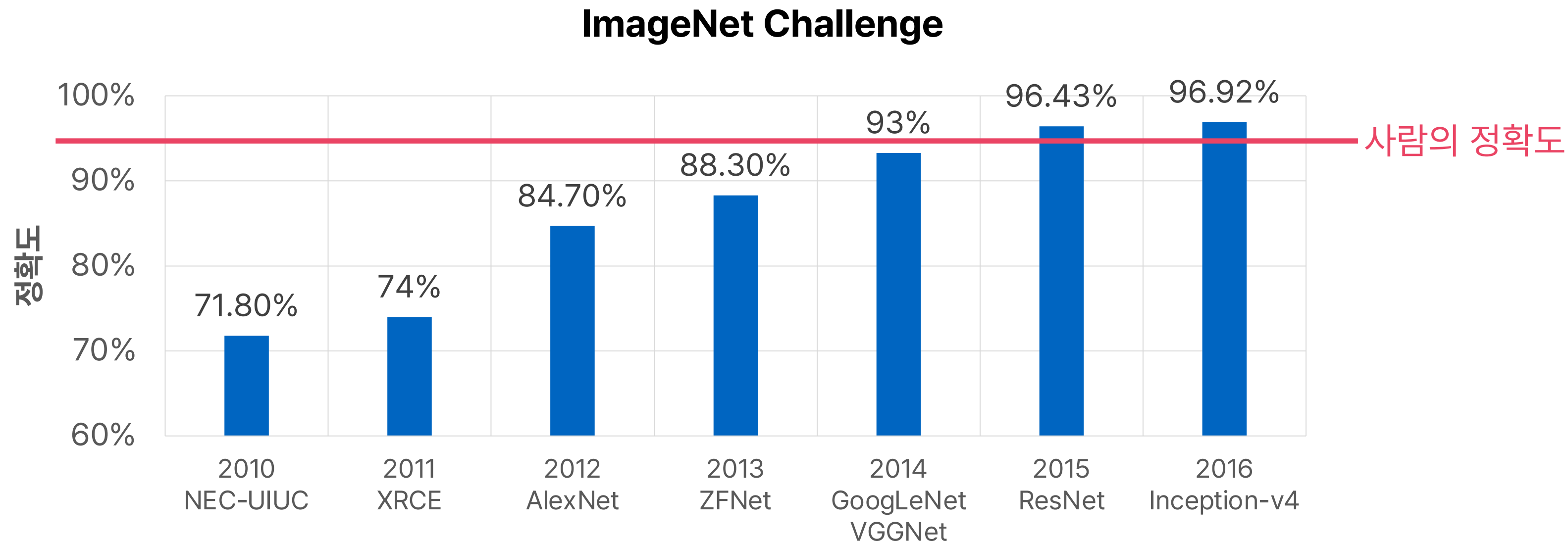
- 이미지를 넣으면 개인지 고양이인지 예측하는 모델
- 이미지를 넣으면 어떤 것의 이미지인지 예측하는 모델
- 이미지를 넣으면 생산품에 이상이 있는지 없는지 예측하는 모델
- ...

④ Large Scale Visual Recognition Challenge (ILSVRC)

인공지능으로 1000가지 종류의 사물과 동물 사진을 분류하는 대회

- ImageNet 이라는 데이터셋을 사용.
- 이미지와 관련된 딥러닝 모델의 발전에 큰 영향을 끼침.

④ 역대 우승 모델들



- 이미지 분류 모델의 기초가 되는 모델들로 간단한 개념을 이해하는 것이 도움이 됨.

01 이미지 분류 모델

☑ 사람의 정확도: 96%?



plow, plough
쟁기



banded gecko
밴디드 게코

- 이런 식으로 1000가지의 물체나 생물이 있음.

☑ AlexNet(2012)

CNN을 본격적으로 사용하기 시작한 모델

- CNN을 통해 특징을 추출하고 분류기 모델로 종류를 예측하는 모델.
- 이후 우승한 모델은 모두 CNN구조를 사용함.

④ CNN(Convolutional Neural Network)

이미지의 특징을 효율적으로 추출할 수 있는 모델 구조

- 2차원 평면의 특징들을 효율적으로 추출하는 모델 구조.
- 성능이 뛰어나서 이미지를 다루는 대부분의 모델이 CNN구조를 포함하고 있음.

☑ VGG16(2014)

모델이 깊을수록 성능이 좋아진다는 것을 발견

- 아주 간단한 구조라도 많은 층을 쌓아서 모델을 만들면 특징을 잘 추출한다는 것을 발견.
- 층은 많이 쌓을수록 성능이 좋다.

☑ ResNet(2015)

깊은 모델도 잘 학습되도록 하는 핵심 방법을 제안

- 모델이 너무 깊어지면 기울기 소실이라는 문제가 발생하면서 학습이 제대로 되지 않는 것을 발견.
- Skip Connection이란 구조를 이용해 아주 깊은 모델도 학습이 잘 되도록 만듦.
- 이 모델부터 인간의 정확도를 뛰어넘음.

☑ Inception-v4(2016)

객체의 크기나 위치에 덜 민감하게 동작하도록 만든 모델

- 사진에서 객체의 위치나 크기에 덜 민감하게 일반적으로 특징을 잘 뽑아낼 수 있는 모델.
- 학습이 효율적으로 되도록 개선.

④ 딥러닝 모델 경량화

가능한 적은 리소스로 높은 성능을 유지하려는 연구

- 이미 성능은 사람보다 훨씬 높은 정확도를 달성함.
- 이제 스마트폰이나 IoT 장비에서 모델을 사용할 수 있도록 경량화하는 연구가 진행됨.

☑ MobileNet

경량화된 이미지 분류 모델의 대표적인 예시

- 연산 횟수와 가중치를 줄여서 모델을 경량화.
- 성능은 여전히 높게 유지하면서 가벼운 모델을 제안.

02

데이터 전처리

④ 이미지 전처리 과정

이미지를 모델에 입력하기 위해 형태와 표현을 변환하는 작업

- 이미지파일을 숫자로 구성된 행렬로 읽어오는 과정.
- 모델에 맞는 크기로 이미지를 자르거나 늘리는 과정.
- 픽셀의 표현 방법을 0~1 또는 -1~1로 변경하는 과정.
- 여러 이미지를 하나의 배열로 묶어주는 과정.

④ 이미지 불러오기

```
from tensorflow.keras.preprocessing import image

img_path = "img1.png"
img = image.load_img(img_path, target_size=(224, 224))
```

- target_size: 원하는 이미지 크기로 변경.
- 이미지의 이름을 전달하면 불러와 PIL.Image객체를 반환.

④ 이미지를 행렬로 바꾸기

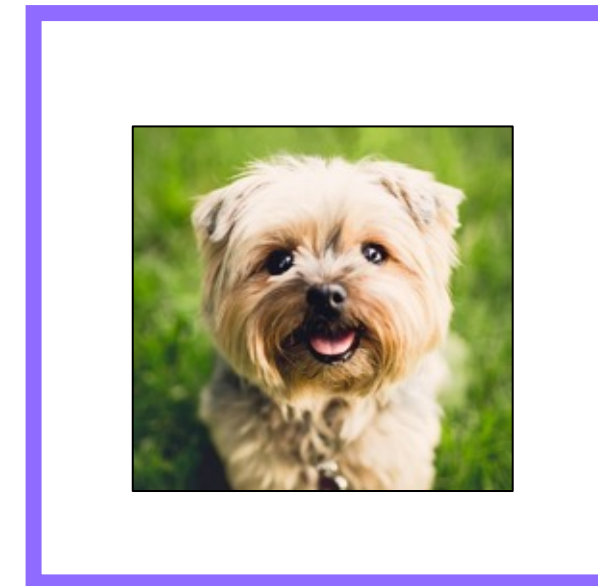
```
# 이미지를 행렬로 바꿔주는 함수  
x = image.img_to_array(img)
```

- 읽어온 이미지를 숫자로 구성된 행렬로 바꾸는 과정.
- 딥러닝 모델은 행렬 연산으로 구성되어 있기 때문에 데이터를 반드시 변환해야 함.

④ 이미지의 차원을 변경

```
# 이미지를 행렬로 바꿔주는 함수  
x = image.img_to_array(img)  
# 이미지를 묶어주는 함수  
x = np.expand_dims(x, axis=0)
```

이미지의 모음으로 묶기



- 대부분 이미지 분류 모델은 여러 개의 데이터를 한번에 입력 받음.
- 우리는 하나의 이미지만 준비했기에 한번 묶어주는 과정이 필요함.
- 이 과정을 차원을 확장한다고 표현.

☑ 모델의 입력과 출력형태

model.summary()

```
Model: "mobilenet_1.00_224"
-----
Layer (type)                Output Shape              Param #
-----
input_1 (InputLayer)        [(None, 224, 224, 3)]     0
conv1 (Conv2D)              (None, 112, 112, 32)      864
...
reshape_2 (Reshape)         (None, 1000)              0
predictions (Activation)    (None, 1000)              0
Total params: 4,253,864
Trainable params: 4,231,976
Non-trainable params: 21,888
-----
```

입력 레이어
형태: [None, 224, 224, 3]
이미지 크기

출력 레이어
형태: [None, 1000]
물체 종류

☑ 모델의 입력과 출력형태

model.summary()

Model: "mobilenet_1.00_224"

Layer (type)	Output Shape	Param #
--------------	--------------	---------

input_1 (InputLayer)	[(None, 224, 224, 3)]	0
----------------------	-----------------------	---

conv1 (Conv2D)	(None, 112, 112, 32)	864
----------------	----------------------	-----

...

reshape_2 (Reshape)	(None, 1000)	0
---------------------	--------------	---

predictions (Activation)	(None, 1000)	0
--------------------------	--------------	---

Total params: 4,253,864

Trainable params: 4,231,976

Non-trainable params: 21,888

입력 레이어

형태: [None, 224, 224, 3]

데이터를 묶어서 입력

출력 레이어

형태: [None, 1000]

결과가 묶여서 나옴

④ 데이터 정규화 (normalization)

0~255인 픽셀 값을 모델에 맞는 특정 범위로 변환하는 작업

- 대부분 0~1 혹은 -1~1로 변환.
- 모델에 따라 사용한 방법이 다르므로 모델이 수행한 방법을 따라가야 가장 좋은 성능이 나타남.

④ 모델이 사용한 정규화 방법을 알아내는 방법

논문을 읽거나 저자의 모델 코드를 보고 구현

- 모델을 제안한 논문에 전처리 단계에 대한 내용을 적어둔 경우 논문을 참고하여 구현.
- 만약 논문에 없다면 저자가 모델을 구현한 코드를 참고하여 구현.
- 유명한 모델은 패키지에 전처리 함수가 포함되어 있음.

☑ Tensorflow에서 제공하는 전처리 함수

```
from tensorflow.keras.applications.mobilenet import (  
    preprocess_input, # mobilenet의 전처리 함수  
    decode_predictions, # mobilenet의 결과를 해석하는 함수  
)  
  
...  
  
x = preprocess_input(x)
```

- 유명한 모델의 경우 tensorflow에서 전처리 함수를 제공.
- 결과를 해석해주는 함수도 함께 제공됨.

④ 전처리 과정 전체

```
from tensorflow.keras.applications.mobilenet import preprocess_input
from tensorflow.keras.preprocessing import image
import numpy as np

def preprocess(img_path: str):
    """이미지의 경로를 입력받아 전처리된 데이터를 반환하는 함수"""

    img = image.load_img(img_path, target_size=(224, 224))
    x = image.img_to_array(img)
    x = np.expand_dims(x, axis=0)
    x = preprocess_input(x)

    return x
```


03

추론 결과 처리하기

④ 분류 모델의 추론

모델에 데이터를 넣어서 결과를 예측하는 과정

- `predict()` 함수에 전처리된 데이터를 입력.
- 반환된 값을 모델에 맞게 해석하여 결과를 얻을 수 있음.
- 모델의 앞에서부터 뒤까지 결과를 전달하기 때문에 순방향 전파라고도 부름

03 추론 결과 처리하기

④ 분류 모델의 추론

```
# 모델에 전처리된 데이터 입력  
preds = model.predict(x)
```

- predict() 함수에 전처리된 데이터를 입력.
- 반환된 값을 모델에 맞게 해석하여 결과를 얻을 수 있음.
- 모델의 앞에서부터 뒤까지 결과를 전달하기 때문에 순방향 전파라고도 부름

03 추론 결과 처리하기

④ 추론 결과

```
[[0.00000017 0.00000006 0.00000001 0.00000001 0.00000001 0.00000018
 0.          0.00008043 0.00000787 0.00000021 0.00000006 0.00000019
0.00000001 0.00000001 0.00000003 0.00000066 0.00000011 0.00000037
0.00000007 0.00000034 0.00000021 0.00000221 0.0000011 0.00000008

...

0.00000013 0.00000021 0.00000004 0.00000041 0.00000009 0.
0.00000002 0.00000016 0.00000039 0.00000061 0.00000097 0.00000009
0.00000009 0.00001642 0.00000032 0.00000015 0.00000153 0.00000302
0.00003129 0.00000045 0.00000076 0.00000007]]
```

- 출력 형태에 맞는 실수들의 행렬이 반환.
- 각 숫자는 해당 클래스일 확률을 나타내는 값.
- 숫자가 크다면 입력한 데이터가 해당 클래스일 확률이 높다는 것을 의미.

03 추론 결과 처리하기

☑ 모델의 입력과 출력형태

model.summary()

```
Model: "mobilenet_1.00_224"
-----
Layer (type)                Output Shape              Param #
-----
input_1 (InputLayer)        [(None, 224, 224, 3)]    0
conv1 (Conv2D)              (None, 112, 112, 32)     864
...
reshape_2 (Reshape)         (None, 1000)             0
predictions (Activation)    (None, 1000)             0
Total params: 4,253,864
Trainable params: 4,231,976
Non-trainable params: 21,888
-----
```

입력 레이어
형태: [None, 224, 224, 3]
이미지 크기

출력 레이어
형태: [None, 1000]
물체 종류

03 추론 결과 처리하기

④ 추론 결과 처리

```
preds = model.predict(x)
top = preds[0].argmax() # 확률이 가장 높은 클래스
print(preds[0][top]) # 확률이 가장 높은 클래스의 확률을 출력
```

- argmax를 통해 가장 큰 값의 인덱스를 계산.
- 가장 확률이 높은 클래스와 그 클래스일 확률을 계산할 수 있음.
- n번째로 확률이 높은 값을 계산하려면 별도의 계산이 필요함.

🔖 용어 해설

argmax()

주어진 배열 또는 시퀀스에서 최댓값을 가진 요소의 인덱스를 반환하는 함수입니다. "arg"는 "인자(argument)"를 의미하고, "max"는 "최댓값(maximum)"을 의미합니다.

03 추론 결과 처리하기

④ 추론 결과 처리

```
from tensorflow.keras.applications.mobilenet import (  
    preprocess_input, # mobilenet의 전처리 함수  
    decode_predictions, # mobilenet의 결과를 해석하는 함수  
)
```

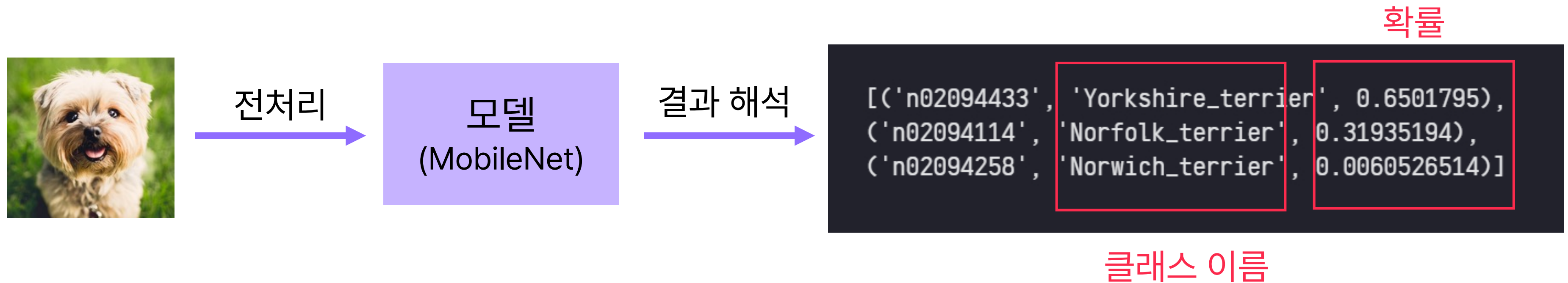
```
preds = model.predict(x) # 모델로 예측  
result = decode_predictions(preds, top=3)[0]  
print(result)
```

```
[('n02094433', 'Yorkshire_terrier', 0.6501795),  
( 'n02094114', 'Norfolk_terrier', 0.31935194),  
( 'n02094258', 'Norwich_terrier', 0.0060526514)]
```

- 가장 확률이 높은 3개의 클래스를 출력.
- tensorflow에서 제공하는 결과를 해석하는 함수 사용

03 추론 결과 처리하기

④ 이미지 분류 모델의 사용과정



- 이미지 파일을 읽어와 전처리하고 모델에 입력.
- 예측한 결과 이 이미지는 65%로 요크셔 테리어라는 예측을 얻음.